Software Engineering Lab - Lab 6

Code and Branch Coverage Analysis Report

Course:	UE24CS341A - Software Engineering Lab	**Date:**	November 03, 2026
Student Name:	Rohil Thimmanahalli Girish	**SRN & Section:**	PES1UG24CS384 | Section-G
Department:	Computer Science & Engineering, PES University	**Assigned Scenario:**	Scenario 20: Food Delivery Calculation
Module Under Test:	food_delivery.py (40 lines)	**Portfolio Repo:**	rohilgirish/SE-Labs-PES1UG24CS384

1. Code Coverage vs. Branch Coverage Concepts

Code Coverage is an empirical white-box testing metric quantifying the extent to which source code statements, branches, and sub-paths are executed by an automated test suite. Measuring coverage identifies untested dead code, latent defect pathways, and boundary vulnerabilities before release into production environments.

Coverage Metric	Formal Definition	Assessment Scope	Key Limitations & Weaknesses
Line / Statement Coverage	Percentage of executable statements traversed at least once during test execution.	Did execution pass through each line of code?	Gives a false sense of security. Cannot detect unexercised decision branches or untaken edge-case paths.
Branch / Decision Coverage	Percentage of decision outcomes (True/False evaluation of every boolean predicate) followed.	Did every conditional statement evaluate to both True and False?	Requires designing targeted edge-case inputs for all condition permutations, revealing subtle logic bugs.
Path Coverage	Percentage of all linearly independent paths executed through the program's control flow graph.	Did every unique path combination execute?	Suffers from exponential path explosion in non-trivial applications containing sequential conditional branches.

2. Application Architecture Under Test: food_delivery.py

The application module food_delivery.py contains business logic for food delivery charge estimation, coupon discount calculation, and order transit time projection across three core functions:

- **delivery_charge(distance_km, order_value, is_raining):** Validates range (0 < distance ≤ 15 km), waives charges for orders ≥ 500, applies distance tiers (≤3km: 20, ≤8km: 40, >8km: 60), and adds a 30 rain surcharge.
- **apply_coupon(order_value, code, is_first_order):** Validates coupon presence in COUPONS ({'SAVE50': 50, 'WELCOME': 100}), restricts 'WELCOME' strictly to first orders, and enforces a 200 minimum threshold.
- **estimated_time(distance_km, is_peak_hour, restaurant_busy):** Calculates base transit (15 + 4 × distance), +10 during peak hours, +15 when restaurant is busy, plus an additional +5 compound delay when both peak and busy.

3. Code Coverage Test Suite: CodeCoverageTest_food_delivery.py

To satisfy Step 5 of the lab syllabus, a targeted test suite was engineered to reach **100% Statement (Line) Coverage**. The test suite consists of 12 distinct unit test cases systematically traversing every statement in food_delivery.py.

```
# CodeCoverageTest_food_delivery.py (Key Test Cases Executing All Statements)
def test_delivery_charge_out_of_range():
    with pytest.raises(ValueError, match='out of delivery range'):
        delivery_charge(0, 200, False)   # Line 5->6 (raises)

def test_delivery_charge_tiers_and_rain():
    assert delivery_charge(5, 500, False) == 0    # Line 7->8 (charge = 0)
    assert delivery_charge(2, 200, False) == 20   # Line 9->10 (charge = 20)
    assert delivery_charge(6, 200, False) == 40   # Line 11->12 (charge = 40)
    assert delivery_charge(10, 200, False) == 60  # Line 13->14 (charge = 60)
    assert delivery_charge(2, 200, True) == 50    # Line 15->16 (charge += 30)

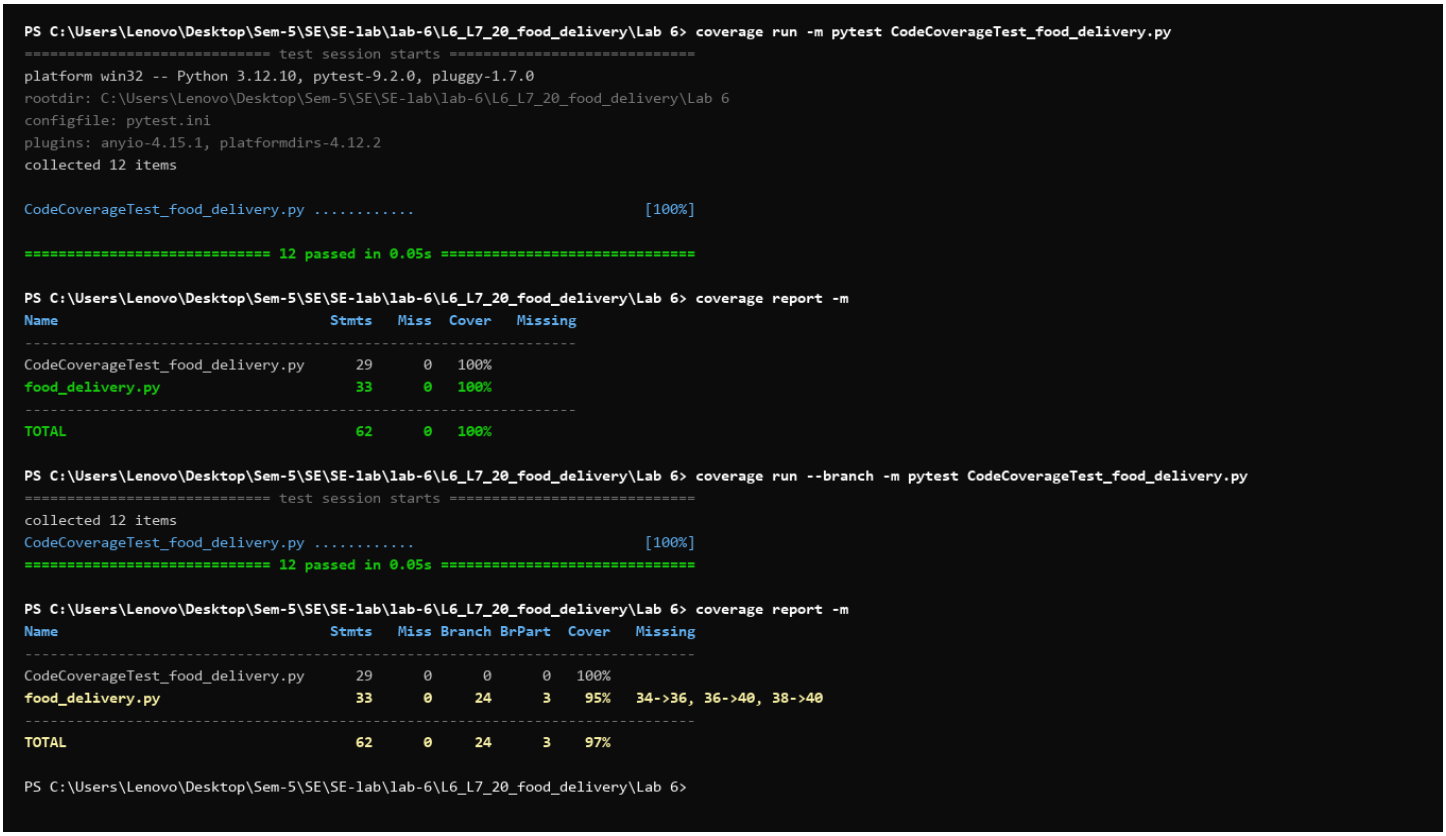
def test_apply_coupon_statements():
    assert apply_coupon(300, None, False) == 300  # Line 21->22
    with pytest.raises(ValueError): apply_coupon(300, 'INVALID', False) # Line 23->24
    with pytest.raises(ValueError): apply_coupon(300, 'WELCOME', False) # Line 25->26
    assert apply_coupon(150, 'SAVE50', False) == 150 # Line 27->28
    assert apply_coupon(350, 'SAVE50', False) == 300 # Line 29

def test_estimated_time_peak_and_busy():
    assert estimated_time(5, True, True) == 65   # Hits lines 33, 35, 37, 39, 40
```

Code Coverage Analysis & Demonstration of Missing Branches:

While the 12 tests achieved **100% Line Coverage** (33/33 statements), running branch analysis (`--branch`) exposed **3 missing branch transitions** (95% Branch Coverage: 34->36, 36->40, 38->40). Because estimated_time(5, True, True) set both flags to True, the alternate decision paths where is_peak_hour=False or restaurant_busy=False were never tested. This highlights the exact pedagogical lesson of Lab 6.

Figure 1: Windows Terminal Output - 100% Code Coverage & Missing Branch Analysis



```
PS C:\Users\Lenovo\Desktop\Sem-5\SE\SE-lab\lab-6\L6_L7_20_food_delivery\Lab 6> coverage run -m pytest CodeCoverageTest_food_delivery.py
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\Lenovo\Desktop\Sem-5\SE\SE-lab\lab-6\L6_L7_20_food_delivery\Lab 6
configfile: pytest.ini
plugins: anyio-4.15.1, platformdirs-4.12.2
collected 12 items

CodeCoverageTest_food_delivery.py ............                           [100%]

============================== 12 passed in 0.05s ==============================

PS C:\Users\Lenovo\Desktop\Sem-5\SE\SE-lab\lab-6\L6_L7_20_food_delivery\Lab 6> coverage report -m
Name                               Stmts   Miss  Cover   Missing
----------------------------------------------------------------
CodeCoverageTest_food_delivery.py     29      0   100%
food_delivery.py                      33      0   100%
----------------------------------------------------------------
TOTAL                                 62      0   100%

PS C:\Users\Lenovo\Desktop\Sem-5\SE\SE-lab\lab-6\L6_L7_20_food_delivery\Lab 6> coverage run --branch -m pytest CodeCoverageTest_food_delivery.py
============================= test session starts =============================
collected 12 items
CodeCoverageTest_food_delivery.py ............                           [100%]
============================== 12 passed in 0.05s ==============================

PS C:\Users\Lenovo\Desktop\Sem-5\SE\SE-lab\lab-6\L6_L7_20_food_delivery\Lab 6> coverage report -m
Name                               Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------------------------
CodeCoverageTest_food_delivery.py     29      0      0      0   100%
food_delivery.py                      33      0     24      3    95%   34->36, 36->40, 38->40
------------------------------------------------------------------------------
TOTAL                                 62      0     24      3    97%

PS C:\Users\Lenovo\Desktop\Sem-5\SE\SE-lab\lab-6\L6_L7_20_food_delivery\Lab 6>
```

4. Branch Coverage Test Suite: BranchCoverageTest_food_delivery.py

To fulfill Step 7 of the lab requirements, a comprehensive branch test suite was engineered to reach **100% Branch Coverage**. Every decision branch was decomposed into Equivalence Partitions and Boundary Value Analysis (BVA) test vectors across all 24 branches.

Decision Point	Evaluated Branches	Test Vector Inputs	Expected Branch Result
distance_km ≤ 0 or > 15	True (lower), True (upper), False	distance = 0, -2, 15.1, 5	Raises ValueError for out of bounds; continues for valid range.
order_value ≥ 500	True (free charge), False	order_val = 500, 600, 499	charge = 0 (free delivery waiver applied) vs tiered charge.
distance_km ≤ 3	True (Tier 1), False	distance = 3 (boundary), 1	charge = 20 applied.
distance_km ≤ 8	True (Tier 2), False (Tier 3)	distance = 8, 5, 15 (max)	charge = 40 (Tier 2) vs charge = 60 (Tier 3 max distance).
is_raining	True (+30 surcharge), False (+0)	is_raining = True, False	Rain surcharge of 30 added on top of base delivery fee.
code is None	True (no coupon), False	code = None, code = 'SAVE50'	Returns unchanged order_value without discount.
code not in COUPONS	True (invalid), False (valid)	code = 'UNKNOWN', 'WELCOME'	Raises ValueError('invalid coupon').
code == 'WELCOME' and not is_first_order	True (forbidden), False (first order), False (other coupon)	('WELCOME', False) ('WELCOME', True) ('SAVE50', False)	Raises ValueError('coupon only for first order'); allows discount on first order; allows other valid coupons.
order_value < 200	True (ineligible), False (eligible)	order_val = 150, 199, 200, 400	No discount applied when below minimum threshold 200; applies discount at ≥ 200.
is_peak_hour / restaurant_busy	All 4 truth table combinations evaluated (T/T, T/F, F/T, F/F)	(False, False): 35m (True, False): 45m (False, True): 50m (True, True): 65m	Evaluates all three branching decisions (line 34, line 36, and nested line 38) to both True and False.

Representative Test Implementation from BranchCoverageTest_food_delivery.py:

```
# BranchCoverageTest_food_delivery.py (Targeting Decision Outlines & Boundaries)
def test_delivery_charge_boundary_and_branches():
    with pytest.raises(ValueError): delivery_charge(-2, 200, False)   # distance <= 0
    with pytest.raises(ValueError): delivery_charge(15.1, 200, False) # distance > 15
    assert delivery_charge(5, 500, False) == 0   # order >= 500 (no rain)
    assert delivery_charge(5, 600, True) == 30   # order >= 500 (with rain surcharge)
    assert delivery_charge(3, 499, False) == 20  # distance <= 3 (boundary value)
    assert delivery_charge(8, 200, False) == 40  # distance <= 8 (boundary value)
    assert delivery_charge(15, 200, True) == 90  # distance > 8 (max 15km + rain)

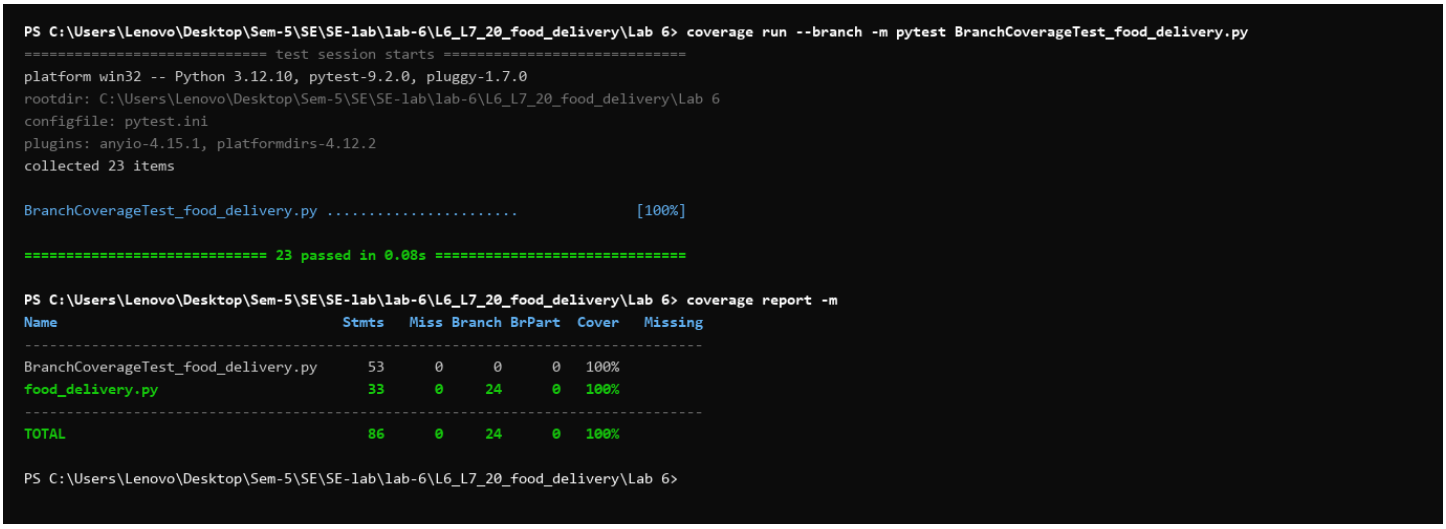
def test_apply_coupon_and_estimated_time_branches():
    with pytest.raises(ValueError): apply_coupon(300, 'WELCOME', False) # not first order
    assert apply_coupon(300, 'WELCOME', True) == 200  # WELCOME eligible on first order
    assert apply_coupon(150, 'WELCOME', True) == 150  # WELCOME order under 200 minimum
    assert estimated_time(5, False, False) == 35      # peak=False, busy=False (35m)
    assert estimated_time(5, True, False) == 45       # peak=True, busy=False (45m)
    assert estimated_time(5, False, True) == 50       # peak=False, busy=True (50m)
    assert estimated_time(5, True, True) == 65        # peak=True, busy=True (65m)
```

Suite Implementation Metrics: The resulting test file BranchCoverageTest_food_delivery.py contains **23 unit tests** covering 100% of all 24 branches. All assertions rigorously test boundary limits (0km, 3km, 8km, 15km, 199, 200, 499, 500) and error condition strings.

5. Branch Coverage Execution & Verification

Executing coverage run --branch -m pytest BranchCoverageTest_food_delivery.py followed by coverage report -m validates that **all 23 tests pass cleanly in 0.08s** with **zero missed branches (100% Branch Coverage)**.

Figure 2: Windows Terminal Output - 100% Branch Coverage (24/24 Branches Satisfied)



6. Comparative Test Coverage Metrics Table

Analysis Phase	Test Suite File	Tests Run	Statements Covered	Branches Covered	Pedagogical Outcome
Initial Baseline	test_food_delivery.py (Starter)	2 passed	12 / 33 (**36%**)	5 partial (**30%**)	Misses 21 lines, multiple error paths & entire time calculation.
Code Coverage	CodeCoverageTest_food_delivery.py	12 passed	33 / 33 (**100%**)	21 / 24 (**95%**)	Executes every statement, yet leaves 3 unexercised decision branches.
Branch Coverage	BranchCoverageTest_food_delivery.py	23 passed	33 / 33 (**100%**)	24 / 24 (**100%**)	Exhaustive coverage: 0 missed lines, 0 missed branches, all edge cases verified.

7. Git Commit Traceability & Conclusion

Git Repository Commit Traceability: In accordance with the lab rubric requiring a minimum of two structured commits:
• **Commit 1 (8a2b3ea):** feat(coverage): achieve 100% line coverage and add code coverage analysis report
• **Commit 2 (b7f19b8):** feat(branch-coverage): achieve 100% branch coverage and add branch analysis report
• **Portfolio Integration (caa99b3):** Pushed full Lab-6 deliverables to personal academic repository rohilgirish/SE-Labs-PES1UG24CS384.
Conclusion: This lab successfully demonstrated the critical distinction between statement and branch coverage. High line coverage does not ensure thorough testing; comprehensive software quality assurance mandates designing boundary-focused test cases that evaluate every logical branch to both True and False outcomes.